TAREA 5

CRISTINA MORENO

1. Sea $f(x) = x\,3 - \cos x$ y $p0 = -1$. Use el método de Newton y de la Secante para encontrar $p2$. ¿Se podría usar $p0 = 0$?

In [39]:
```
import numpy as np
from scipy.optimize import newton

def f(x):
    return x**3-np.cos(x)

def fprime(x):
    return 3*x**2+np.sin(x)

x0=-1
resultadoNewton1=newton(f,fprime=fprime,x0=x0)
print("Raiz encontrada con newton {resultadoNewton1}")

resultadoSecante1=newton(f,x0=-1,x1=0)
print("Raiz encontrada con secante{resultadoSecante1}")

x1=0
x2=1
print("Con x0=0")
try:
    resultadoNewton2 = newton(f, fprime=fprime, x0=x1)
    print("Raiz encontrada con Newton: {resultadoNewton2}")
except RuntimeError:
    print("No converge con Newton con el punto inicial {x1}")


try:
    resultadoSecante2=newton(f,x0=x1,x1=x2)
    print("Raiz encontrada con secante{resultadoSecante2}")
except RuntimeError:
    print("No converge con secante con los puntos iniciales {x1,x2}")
```

```
Raiz encontrada con newton -0.8654760331854144
Raiz encontrada con secante-0.8654760331854144
Con x0=0
No converge con Newton con el punto inicial 0
No converge con secante con los puntos iniciales {0, 1}
```

2. Encuentre las soluciones precisas para 10^{-6}

a. $x^3 - 2x^2 - 5 = 0$, $[1, 4]$

b. $x^3 + 3x^2 - 1 = 0$, $[-3, -2]$

c. $x - \cos(x) = 0$, $\left[0, \frac{\pi}{2}\right]$

d. $x - 0.8 - 0.2\sin(x) = 0$, $\left[0, \frac{\pi}{2}\right]$

Como el ejercicio nos da dos puntos es preferible usar el metodo de la secante

In [12]:
```
def f1(x):
    return x**3-2*x**2-5
def f2(x):
    return x**3+3*x**2-1
def f3(x):
    return x-np.cos(x)
def f4(x):
    return x-0.8-0.2*np.sin(x)

try:
    resultadoSecante1=newton(f5,x0=1,x1=4,tol=10**-6)
    print("x Raiz encontrada con secante {resultadoSecante1}")
    resultadoSecante2=newton(f2,x0=-3,x1=-2,tol=10**-6)
    print("b Raiz encontrada con secante {resultadoSecante2}")
    resultadoSecante3=newton(f3,x0=0,x1=np.pi/2,tol=10**-6)
    print("c Raiz encontrada con secante {resultadoSecante3}")
    resultadoSecante4=newton(f4,x0=0,x1=np.pi/2,tol=10**-6)
    print("d Raiz encontrada con secante {resultadoSecante4}")
except RuntimeError as e:
    print("No converge: {e}")
```

```
a Raiz encontrada con secante 2.690647447837734
b Raiz encontrada con secante -1.879385147548809
c Raiz encontrada con secante 0.7390851332964638
d Raiz encontrada con secante 0.9641338557963112
```

3. Encuentre soluciones por los dos metodos para 10^{-5}

$a3x - e^x = 0, \quad 1 \le x \le 2$

$b2x + 3\cos(x) - e^x = 0, \quad 1 \le x \le 2$

In [13]:
```
def f5(x):
    return 3*x-np.exp(x)

def f6(x):
    return 2*x+3*np.cos(x)-np.exp(x)

def fprime(x):
    return 3-np.exp(x)
def fprime(x):
    return 2-2*np.sin(x)-np.exp(x)
try:
    print("Raices con Newton"
    resultadoNewton3=newton(f5,fprime=fprime,x0=2,tol=10**-5)
    print("Raiz: {resultadoNewton3}")
    resultadoNewton4=newton(f6,fprime=fprime,x0=2,tol=10**-5)
    print("Raiz: {resultadoNewton4}")
    print("Raices con Secante")
    resultadoSecante7=newton(f5,x0=1,x1=2,tol=10**-5)
    print("Raiz: {resultadoSecante7}")
    resultadoSecante8=newton(f6,x0=1,x1=2,tol=10**-5)
    print("Raiz: {resultadoSecante8}")
except RuntimeError as e:
    print("No converge: {e}")
```

```
Raices con Newton
Raiz: 1.2309594319527
Raiz: 1.2309594319527176
Raices con Secante
Raiz: 1.5213301518062
Raiz: 1.2309594319527154
```

4. El polinomio de cuarto grado es:

$f(x) = 230x^4 + 18x^3 + 9x^2 - 221x - 9$

Queremos encontrar las raices de $f(x) = 0$ en los intervalos $[-1, 0]$ y $[0, 1]$, con una aproximación de precisión 10^{-8}.

a) El metodo de la secante (use los extremos como las estimaciones iniciales)

In [15]:
```
def f(x):
    return 230*x**4+18*x**3+9*x**2-221*x-9

resultadoSecante9=newton(f,x0=-1,x1=0,tol=10**-6)
print("Resultado cuando [-1,0]: {resultadoSecante9}")

resultadoSecante10=newton(f,x0=0,x1=1,tol=10**-6)
print("Resultado cuando [0,1]: {resultadoSecante10}")
```

```
Resultado cuando [-1,0]: -0.040619288315575
Resultado cuando [0,1]: -0.040619288315572
```

Para conseguir la otra raiz deberia modificar el intervalo

In [ ]:
```
resultadoBid1=newton(f,x0=0.5,x1=1,tol=10**-6)
print("Resultado con otro intervalo de [0,5,1]: {resultadoBid1}")
```

```
Resultado con otro intervalo de [0.2,1]: 0.962398418750658
```

b. El metodo de Newton (use el punto medio como estimación inicial)

In [20]:
```
def f(x):
    return 230*x**4+18*x**3+9*x**2-221*x-9
def fprime(x):
    return 920*x**3+54*x**2+18*x-221

resultadoNewton5=newton(f,x0=-0.5,fprime=fprime,tol=10**-6)
print("Resultado cuando el punto medio es -0.5: {resultadoNewton5}")

resultadoNewton6=newton(f,x0=0.5,fprime=fprime,tol=10**-6)
print("Resultado cuando el punto medio es 0.5: {resultadoNewton6}")
```

```
Resultado cuando el punto medio es -0.5: -0.040619288315575899
Resultado cuando el punto medio es 0.5: -0.040619288315575865
```

Igual que en de la secante si de modifica el punto medio se obtiene la otra raiz

In [20]:
```
resultadoBid2=newton(f,x0=0.6,fprime=fprime,tol=10**-6)
print("Resultado con otro punto medio 0.6: {resultadoBid2}")
```

```
Resultado con otro punto medio 0.6: 0.962398418750916
```

5. La función $f(x) = \tan(\pi x) - 6$ tiene un cero en $\left(\frac{1}{3}\right)$. donde aoctaa(6) ≈ 0.44743153. Sea $p_0 = 0$ y $p_1 = 0.48$, y use 10 iteraciones en cada uno de los siguientes metodos para aproximar esta raíz.

¿Cuál metodo es más eficaz y por qué?

a. *Metodo de Bisección*

b. *Metodo de Newton*

c. *Metodo de la Secante*

In [34]:
```
import numpy as np
from scipy.optimize import bisect, newton

def f(x):
    return np.tan(np.pi*x) - 6

def fprime(x):
    return np.pi / (np.cos(np.pi*x)**2

p0 = 0
p1 = 0.48
p0 = 0.24
iteraciones = 10

try:
    raiz_bisect = bisect(f, p0, p1, maxiter=10)
    print("Raiz con bisección:{raiz_bisect}")
except RuntimeError as e:
    print("No converge con bisección: {e}")
```

```
No converge con Bisección: Failed to converge after 10 iterations.
```

In [40]:
```
import numpy as np
from scipy.optimize import bisect, newton

def f(x):
    return np.tan(np.pi*x) - 6

def fprime(x):
    return np.pi / (np.cos(np.pi*x)**2

p0 = 0
p1 = 0.48
p0 = 0.24
iteraciones = 10

try:
    raizNewton = newton(f,fprime=fprime,x0=p0, maxiter=10)
    print("Raiz con newton:{raizNewton}")
except RuntimeError as e:
    print("No converge con newton: {e}")
```

```
No converge con newton: Failed to converge after 10 iterations, value is 13.655812218663435.
```

In [43]:
```
import numpy as np
from scipy.optimize import bisect, newton

def f(x):
    return np.tan(np.pi*x) - 6

def fprime(x):
    return np.pi / (np.cos(np.pi*x))**2

p0 = 0
p1 = 0.48
iteraciones = 10

try:
    raizsecante = newton(f,x0=p0, x1=p1, maxiter=10)
    print("Raiz con secante:{raizsecante}")
except RuntimeError as e:
    print("No converge con secante: {e}")
```

```
No converge con secante: Failed to converge after 10 iterations, value is -3694.35808967476.
```

Ninguno Converge debido al minimo de iteraciones ningun metodo alcanza

6. La función descrita por

$f(x) = \ln(x^2 + 1) - e^{0.4x}\cos(\pi x)$

tiene un número infinito de ceros.

a. Determine, dentro de 10−6, el único cero negativo.

In [48]:
```
import numpy as np
from scipy.optimize import newton

def f(x):
    return np.log(x**2 + 1) - np.exp(0.4*x) * np.cos(np.pi*x)

x0 = -1
x1 = 0
tolerancia = 1e-6

raiz_secante = newton(f, x0=x0, x1=x1, tol=tolerancia)
print("El cero negativo aproximado usando secante es x = {raiz_secante}")
```

```
El cero negativo aproximado usando secante es x = -0.43414304728735
```

b. Determine, dentro de 10−6 , los cuatro ceros positivos más pequeños

In [49]:
```
raiz_secante1 = newton(f, x0=0, x1=0.5, tol=tolerancia)
print("El 1ro aproximado usando secante es x = {raiz_secante1}")
raiz_secante2 = newton(f, x0=1, x1=2, tol=tolerancia)
print("El 2do aproximado usando secante es x = {raiz_secante2}")
raiz_secante3 = newton(f, x0=2, x1=2.5, tol=tolerancia)
print("El 3ro aproximado usando secante es x = {raiz_secante3}")
raiz_secante4 = newton(f, x0=3, x1=4, tol=tolerancia)
print("El 4to aproximado usando secante es x = {raiz_secante4}")
```

```
El 1ro aproximado usando secante es x = 0.45861473789513
El 2do aproximado usando secante es x = 3.74473985356868
El 3ro aproximado usando secante es x = 2.23613707596851126
El 4to aproximado usando secante es x = 3.70904128513965516
```

c. Determine una aproximación inicial razonable para encontrar el enésimo cero positivo más pequeño de f

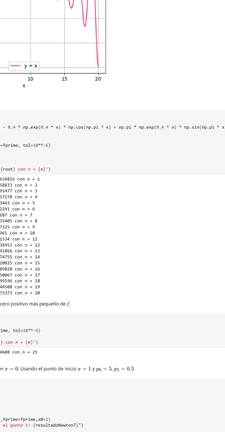
In [50]:
```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-2, 20, 400)

y1 = np.log(x**2 + 1) - np.exp(0.4*x) * np.cos(np.pi*x)

plt.figure(figsize=(6, 5))
plt.plot(x, y1, label= y = ', color="#F02075")

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```



Usamos de numero razonable 20

In [54]:
```
roots = []
def f(x):
    return 0.5 * x / x ** 2 + 1) - 0.4 * np.exp(0.4 * x) * np.cos(np.pi * x) + np.pi * np.exp(0.4 * x) * np.sin(np.pi * x)
for n in range(1, 21):
    p0 = n - 0.5
    ans = newton(f, x0=p0, fprime=fprime, tol=10**-6)
    roots.append(n, ans))

for n, root in roots:
    print(f"Raiz aproximada: p = {root} con n = {n}")
```

```
Raiz aproximada: p = 0.45861473789513016 con n = 1
Raiz aproximada: p = 1.7447378997538833 con n = 2
Raiz aproximada: p = 2.2361370759575126 con n = 3
Raiz aproximada: p = 3.7089412851965516 con n = 4
Raiz aproximada: p = 4.5226498153344387 con n = 5
Raiz aproximada: p = 5.6195354587128 con n = 6
Raiz aproximada: p = 6.4803363278468T con n = 7
Raiz aproximada: p = 7.5632185218345 con n = 8
Raiz aproximada: p = 8.4348097917525 con n = 9
Raiz aproximada: p = 9.5048188125 con n = 10
Raiz aproximada: p = 10.4715765872174 con n = 11
Raiz aproximada: p = 11.4378588187112 con n = 12
Raiz aproximada: p = 12.4058364279384 con n = 13
Raiz aproximada: p = 13.5074724851755 con n = 14
Raiz aproximada: p = 14.4948189185188 con n = 15
Raiz aproximada: p = 15.5053913489968 con n = 16
Raiz aproximada: p = 16.4911665149 con n = 17
Raiz aproximada: p = 17.5014614799590 con n = 18
Raiz aproximada: p = 18.4988152894698 con n = 19
Raiz aproximada: p = 19.5007749322273 con n = 20
```

dubentro de 10−6, el vigesimoquinto cero positivo más pequeño de f.

In [57]:
```
n=25
p0 = n - 0.5
ans = newton(f, x0=p0, fprime=fprime, tol=10**-6)

print("Raiz aproximada: p = {ans} con n = {n}")

Raiz aproximada: p = 24.4998874744608 con n = 25
```

7. La función $f(x) = \sqrt[3]{x}$ tiene raíz en $p = 0$. Usando el punto de inicio $p = 1$ y $p_0 = 5$, $p_1 = 0.5$

In [39]:
```
def f(x):
    return x**(1/3)

def fprime(x):
    return 1/3*x**-2/3
try:

    resultadoNewton1=newton(f,fprime=fprime,x0=1)
    print("Raiz con newton y el punto 1: {resultadoNewton7}")

except RuntimeError as e:
    print("Error al converger:",e)
```

```
Error al converger Failed to converge after 50 iterations, value is nan.
C:\Users\omoreno\AppData\Local\Temp\ipykernel_18840\4141542.py:2: RuntimeWarning: invalid value encountered in scalar power
  return x**(1/3)
```

In [39]:
```
def f(x):
    return x**(1/3)

try:

        resultadoSecante11=newton(f,x0=5,x1=0.5)
        print("Raiz con secante con los puntos 5 y 0,5: {resultadoSecante11} ")
except RuntimeError as e:
    print("Error al converger:",e)
```

```
Error al converger Failed to converge after 50 iterations, value is nan.
C:\Users\omoreno\AppData\Local\Temp\ipykernel_18840\1711542.py:2: RuntimeWarning: invalid value encountered in scalar power
  return x**(1/3)
```

Ninguno tiene convergencia debido a que esta función solo tiene una raíz y no se esta analizando en ninguna ese punto que es en cero